

Tài liệu chi tiết cho dự án RAE XLA và JAX

Nhánh TorchXLA/TPU và lớp tương thích JAX/NNX của dự án Representation Autoencoders

Ngày 19 tháng 3 năm 2026

Mục lục

1	Tổng quan dự án	3
1.1	Mục tiêu	3
1.2	Hai giai đoạn chính	3
1.2.1	Stage 1	3
1.2.2	Stage 2	3
1.3	Phạm vi của branch hiện tại	3
1.4	Giới hạn hiện tại	3
1.5	Dòng dữ liệu cấp cao	4
1.6	Ý tưởng của đường JAX	4
1.7	Tài liệu trong repo	4
2	Kiến trúc hệ thống	4
2.1	Bản đồ codebase	4
2.2	Stage 1: Representation Autoencoder	5
2.3	Stage 2: DiT trong không gian latent	5
2.4	Transport objective	5
2.5	Lớp adapter JAX	6
2.6	Hai mô hình thực thi	6
2.6.1	TorchXLA	6
2.6.2	JAX / NNX	6
2.7	Checkpoint và tương thích trọng số	7
3	Workflow thực chiến	7
3.1	Cài đặt môi trường	7
3.1.1	Đường XLA	7
3.1.2	Phần bổ sung cho JAX / NNX	7
3.2	Chuẩn bị model và dữ liệu	7
3.2.1	Model	7
3.2.2	Dữ liệu	7
3.3	Kiểm tra Stage 1 bằng reconstruction	8
3.3.1	Đường XLA / PyTorch	8
3.3.2	Đường JAX	8
3.3.3	Notebook Kaggle cho TPU v5e-8	8
3.4	Huấn luyện Stage 2 trên TPU bằng TorchXLA	8
3.5	Huấn luyện Stage 2 bằng JAX / NNX	9

3.6	Wandb logging	9
3.7	FID trong train loop	9
3.8	Sampling Stage 2	9
3.8.1	JAX một lệnh	9
3.8.2	JAX distributed	10
3.9	Đẩy artifact lên Hugging Face	10
3.9.1	Lệnh độc lập	10
3.9.2	Tích hợp ngay trong train	10
4	Cấu hình và ánh xạ sang JAX	10
4.1	Các block config chính	10
4.2	Ma trận script và block config	10
4.3	Block stage_1	11
4.4	Block stage_2	11
4.5	Block guidance	11
4.6	Block training	11
4.7	Block eval	12
4.8	Override từ CLI	12
5	Vận hành, artifact và FID	12
5.1	Sampling Stage 2 trên đường JAX	12
5.1.1	Sampling nhanh	12
5.1.2	Sampling phân tán	12
5.2	Build reference statistics cho FID	13
5.3	Artifact của Stage 2 training	13
5.3.1	Đường XLA	13
5.3.2	Đường JAX	13
5.4	Đẩy model lên Hugging Face	13
5.5	Các lưu ý vận hành quan trọng	14
5.6	Checklist thực tế trước khi chạy dài	14

1. Tổng quan dự án

1.1. Mục tiêu

Dự án này triển khai pipeline tạo ảnh hai giai đoạn dựa trên *Representation Autoencoders* (RAE). Ở thời điểm hiện tại, repo có hai đường chạy chính:

- `src/`: nhánh TorchXLA, tối ưu cho TPU với train và sampling Stage 2, reconstruction Stage 1, và FID phía host;
- `src_jax/`: lớp tương thích JAX/NNX, giữ nguyên schema YAML của repo nhưng ánh xạ sang backend NNX để tránh nhân đôi toàn bộ codebase.

1.2. Hai giai đoạn chính

1.2.1. Stage 1

Stage 1 dùng một encoder biểu diễn đã được pretrain và đóng băng, kết hợp với một decoder ViT để tái tạo ảnh. Đầu ra của Stage 1 là latent giàu ngữ nghĩa hơn latent VAE thông thường.

1.2.2. Stage 2

Stage 2 học mô hình diffusion transformer trong không gian latent của Stage 1. Thay vì học trực tiếp trên pixel, mô hình học quy luật biến đổi của latent, sau đó giải mã latent sinh được thành ảnh RGB qua decoder của RAE.

1.3. Phạm vi của branch hiện tại

Nhánh hiện tại không chỉ có đường TorchXLA mà còn có adapter JAX mỏng. Phần được hỗ trợ thực dụng nhất là:

- huấn luyện Stage 2 bằng `src/train.py` trên TPU;
- huấn luyện Stage 2 bằng `src_jax/train.py` trên JAX/NNX;
- sampling Stage 2 bằng cả `src/` và `src_jax/`;
- reconstruction Stage 1 để kiểm tra encoder-decoder trên cả hai đường;
- logging qua wandb;
- upload artifact hoặc workdir lên Hugging Face;
- FID offline, và online FID trong train loop ở đường XLA.

1.4. Giới hạn hiện tại

- `src/` vẫn không có endpoint riêng cho huấn luyện Stage 1 trên XLA;
- `src_jax/` cố ý chưa port toàn bộ vòng huấn luyện Stage 1 kiểu GAN + LPIPS + discriminator, vì phần đó sẽ làm codebase lớn và khó bảo trì hơn đáng kể.

1.5. Dòng dữ liệu cấp cao

Bước	Mô tả
1	Ảnh đầu vào được đưa qua encoder của Stage 1
2	Encoder sinh latent tensor có kích thước như <code>misc.latent_size</code>
3	Stage 2 học transport objective trên latent đó
4	Khi sampling, latent được sinh bởi Stage 2
5	Decoder của Stage 1 giải mã latent thành ảnh RGB

1.6. Ý tưởng của đường JAX

Đường JAX không tạo thêm một schema config mới. Thay vào đó:

- vẫn đọc các block `stage_1`, `stage_2`, `transport`, `sampler`, `guidance`, `misc`, `training`, `eval`;
- dùng `src_jax/config_adapter.py` để chuyển YAML hiện có sang config mà backend NNX hiểu được;
- bootstrap backend `diffuse_nnx` theo commit cố định để branch gọn hơn, không phải chép toàn bộ framework vào repo.

1.7. Tài liệu trong repo

Ngoài PDF này, repo còn có bộ tài liệu markdown trong `docs/`. Phần markdown thiên về runbook ngắn và tra cứu nhanh; PDF này gom lại kiến trúc, workflow, cấu hình và vận hành cho cả đường XLA lẫn JAX.

2. Kiến trúc hệ thống

2.1. Bản đồ codebase

```
configs/
  stage1/
  stage2/
src/
  stage1/
  stage2/
  utils/
  train.py
  sample.py
  sample_ddp.py
  stage1_sample.py
  stage1_sample_ddp.py
  build_fid_stats.py
  evaluate_fid.py
src_jax/
  vendor.py
  config_adapter.py
  stage2_runtime.py
  stage1_runtime.py
  hf_utils.py
  train.py
  sample.py
```

```
sample_ddp.py
stage1_sample.py
push_hf.py
```

2.2. Stage 1: Representation Autoencoder

Thành phần gốc của Stage 1 nằm trong `src/stage1/rae.py`. Lớp RAE kết hợp:

- encoder ở `src/stage1/encoders/`,
- decoder ở `src/stage1/decoders/`,
- latent normalization qua `normalization_stat_path`,
- latent noising qua `noise_tau`.

Hành vi chính:

- `encode(x)` resize đầu vào về đúng độ phân giải của encoder, chuẩn hóa theo image processor, chạy encoder, reshape latent về dạng 2D nếu cần, rồi áp normalization;
- `decode(z)` đảo normalization, đổi latent về dạng sequence nếu cần, rồi giải mã thành ảnh RGB bằng decoder ViT.

Ở đường JAX, `src_jax/stage1_runtime.py` không tự viết lại toàn bộ RAE trong repo này. Nó gọi backend NNX, nhưng vẫn dùng đúng các giá trị cấu hình trong block `stage_1`.

2.3. Stage 2: DiT trong không gian latent

Implementation PyTorch/XLA chính nằm trong `src/stage2/models/DDT.py`. Mô hình xuất hiện nhiều nhất trong repo này là `DiTwDDTHead`.

Các đặc điểm quan trọng:

- đầu vào là latent thay vì pixel;
- embedding thời gian bằng Gaussian Fourier features;
- class conditioning bằng `LabelEmbedder`;
- tùy chọn ROPE, RMSNorm, SwiGLU, qk-norm;
- hỗ trợ classifier-free guidance và autoguidance khi sampling.

Đường JAX ánh xạ `stage_2.target` sang backbone tương ứng của backend NNX:

- `DiTwDDTHead` → `lightning_ddt`,
- `LightningDiT` → `lightning_dit`,
- các target kiểu DiT khác → `dit`.

2.4. Transport objective

Phần này nằm trong `src/stage2/transport/transport.py`. Đây là lớp trung gian giữa latent diffusion model và train loop.

Nhiệm vụ của transport:

- lấy mẫu thời gian t ;
- xây dựng đường nối giữa noise và dữ liệu thật;
- tính target phù hợp với loại output của model;
- trả về drift function để phục vụ ODE hoặc SDE sampler.

Ở đường JAX, adapter không giữ lại implementation transport gốc này. Nó ánh xạ các ý nghĩa tương đương sang interface của backend NNX, chủ yếu là SiT với sampling kiểu Euler.

2.5. Lớp adapter JAX

File	Vai trò
<code>src_jax/vendor.py</code>	Bootstrap <code>diffuse_nnx</code> vào ■ <code>/.cache/rae_jax/diffuse_nnx</code> và pin commit
<code>src_jax/config_adapter.py</code>	Chuyển YAML OmegaConf của repo sang config backend NNX
<code>src_jax/stage2_runtime.py</code>	Train, sampling, load checkpoint PyTorch hoặc Orbax, FID glue, wandb bridge
<code>src_jax/stage1_runtime.py</code>	Reconstruction Stage 1 trên đường JAX
<code>src_jax/hf_utils.py</code>	Upload file hoặc thư mục lên Hugging Face

2.6. Hai mô hình thực thi

2.6.1. TorchXLA

Các entrypoint distributed trong `src/` dùng `xmp.spawn(...)` và mỗi TPU core chạy một process riêng. Các primitive quan trọng:

- `DistributedSampler`,
- `ParallelLoader`,
- `xm.optimizer_step(...)`,
- `xm.rendezvous(...)` và `xm.mesh_reduce(...)`.

2.6.2. JAX / NNX

Đường `src_jax/` không tự quản lý toàn bộ graph, checkpoint và sampler ở mức thấp. Nó dựa vào backend NNX để:

- tạo model, optimizer, sampler và EMA,
- quản lý checkpoint Orbax,
- chạy sampling và FID trong định dạng tensor NHWC của JAX,
- mở rộng sang nhiều process JAX nếu môi trường đã cấu hình phù hợp.

2.7. Checkpoint và tương thích trọng số

Một điểm quan trọng của nhánh JAX là vẫn tận dụng được checkpoint cũ:

- nếu `stage_2.ckpt` là file PyTorch `.pt`, adapter sẽ port state dict sang NNX để inference hoặc khởi tạo train;
- nếu `stage_2.ckpt` là thư mục checkpoint Orbx, adapter sẽ restore trực tiếp run JAX trước đó;
- decoder Stage 1 ở đường JAX vẫn dùng được checkpoint decoder PyTorch.

3. Workflow thực chiến

3.1. Cài đặt môi trường

Repo hiện có hai cụm phụ thuộc chính.

3.1.1. Đường XLA

```
conda create -n rae python=3.10 -y
conda activate rae
pip install uv
uv pip install torch~=2.5.0 torch_xla[tpu]~=2.5.0 torchvision==0.20.1 -f https://
    storage.googleapis.com/libtpu-releases/index.html
uv pip install timm==0.9.16 accelerate==0.23.0 torchdiffeq==0.2.5 wandb scipy torch-
    fidelity
uv pip install "numpy<2" transformers einops
```

3.1.2. Phần bổ sung cho JAX / NNX

```
uv pip install "jax[cuda12]==0.5.1" flax==0.10.4 optax==0.2.4 orbax-checkpoint==0.11.16
uv pip install ml-collections clu absl-py etils huggingface_hub
```

Lần chạy JAX đầu tiên sẽ tự bootstrap backend `diffuse_nnx` vào thư mục `./.cache/rae_jax/diffuse_nnx`.

3.2. Chuẩn bị model và dữ liệu

3.2.1. Model

```
hf download nyu-visionx/RAE-collections --local-dir models
```

3.2.2. Dữ liệu

Hầu hết các script dùng dữ liệu dạng `ImageFolder`. Ví dụ:

```
dataset_root/
  class_a/
    0001.png
  class_b/
    0002.png
```

3.3. Kiểm tra Stage 1 bằng reconstruction

3.3.1. Đường XLA / PyTorch

```
python src/stage1_sample.py \  
  --config <config> \  
  --image assets/pixabay_cat.png \  
  --output recon.png
```

3.3.2. Đường JAX

```
python3 src_jax/stage1_sample.py \  
  --config <config> \  
  --image assets/pixabay_cat.png \  
  --output recon_jax.png
```

Lệnh này phù hợp để xác nhận:

- checkpoint decoder Stage 1 có load được hay không,
- latent shape có đúng hay không,
- preprocessing của encoder có khớp hay không.

3.3.3. Notebook Kaggle cho TPU v5e-8

Nếu chạy trên Kaggle TPU v5e-8, dùng notebook `raes-jax-celeba-kaggle-tpuv5e8.ipynb`. Notebook này cài `jax[tpu]` và kiểm tra `jax/TPU` trong một tiến trình Python mới để tránh lỗi khi vừa reinstall `jax/jaxlib` ngay trong notebook.

3.4. Huấn luyện Stage 2 trên TPU bằng TorchXLA

```
python src/train.py \  
  --config configs/stage2/training/ImageNet256/DiTDH-XL_DINOv2-B.yaml \  
  --data-path <imagenet_train_split> \  
  --results-dir results \  
  --image-size 256 \  
  --precision bf16
```

Trong mỗi optimizer step, train loop thực hiện:

1. center crop và đưa ảnh thành tensor;
2. mã hóa ảnh bằng RAE;
3. tính transport loss trong latent space;
4. step optimizer và scheduler trên TPU;
5. update EMA;
6. chạy logging, checkpointing, preview sample, validation và FID theo cadence.

3.5. Huấn luyện Stage 2 bằng JAX / NNX

```
python3 src_jax/train.py \
  --config configs/stage2/training/ImageNet256/DiTDH-XL_DINOv2-B.yaml \
  --data-path <imagenet_train_root> \
  --results-dir results_jax \
  --precision bf16 \
  --wandb
```

Điểm quan trọng:

- vẫn dùng cùng file YAML của repo;
- hỗ trợ `-set key=value` để override nhanh từ CLI;
- nếu `stage_2 ckpt` là file PyTorch `.pt`, adapter sẽ port trọng số sang NNX để khởi tạo;
- nếu `stage_2 ckpt` là thư mục Orbax, adapter sẽ restore run JAX trước đó.

3.6. Wandb logging

Biến môi trường khuyến nghị:

```
export ENTITY=<wandb_entity>
export PROJECT=<wandb_project>
export WANDB_KEY=<wandb_api_key>
```

Đường JAX cũng chấp nhận các tên `WANDB_ENTITY` và `WANDB_API_KEY`. Adapter sẽ bridge các tên legacy ở trên nếu chúng đã được export sẵn.

3.7. FID trong train loop

Đường XLA hỗ trợ online FID trong `src/train.py`. Ví dụ:

```
eval:
  fid_ref: /path/to/reference_stats.npz
  fid_every: 25000
  fid_num_samples: 4096
  fid_per_proc_batch_size: 4
  fid_batch_size: 128
  fid_device: cpu
  fid_num_threads: 96
```

Đường JAX hiện ánh xạ phần FID quan trọng nhất của block `eval`, nhưng chưa có parity hoàn toàn cho validation loss như đường XLA.

3.8. Sampling Stage 2

3.8.1. JAX một lệnh

```
python3 src_jax/sample.py \
  --config configs/stage2/sampling/ImageNet256/DiTDHXL-DINOv2-B_AG.yaml \
  --class-labels 207,360 \
  --output sample_jax.png
```

3.8.2. JAX distributed

```
python3 src_jax/sample_ddp.py \
  --config configs/stage2/sampling/ImageNet256/DiTDHXL-DINOv2-B.yaml \
  --sample-dir samples_jax \
  --num-samples 50000 \
  --label-sampling equal \
  --fid-ref /path/to/reference_stats.npz
```

3.9. Đẩy artifact lên Hugging Face

3.9.1. Lệnh độc lập

```
python3 src_jax/push_hf.py \
  --path results_jax/<run_name> \
  --repo-id <hf_user_or_org>/<repo_name>
```

3.9.2. Tích hợp ngay trong train

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

4. Cấu hình và ánh xạ sang JAX

4.1. Các block config chính

Tất cả entrypoint quan trọng đều dựa trên YAML của repo. Các block top-level:

- stage_1,
- stage_2,
- transport,
- sampler,
- guidance,
- misc,
- training,
- eval.

4.2. Ma trận script và block config

Block	src_jax/train.py	src_jax/sample.py	src_jax/sample_ddp.py	stage1_sample.py
stage_1	yes	yes	yes	yes
stage_2	yes	yes	yes	no
transport	yes	yes	yes	no
sampler	yes	yes	yes	no

Block	src_jax/train.py	src_jax/sample.py	src_jax/sample.py	src_jax/sample.py
guidance	yes	yes	yes	no
misc	yes	yes	yes	no
training	yes	no	no	no
eval	yes, một phần	no	no	no

4.3. Block stage_1

Adapter JAX không tạo schema mới cho Stage 1. Nó đọc trực tiếp:

- `encoder_config_path` hoặc `encoder_params.dinov2_path`,
- `pretrained_decoder_path`,
- `normalization_stat_path`,
- `encoder_input_size`.

Các giá trị này được chuyển sang encoder RAE của backend NNX để phục vụ reconstruction và decode latent khi sampling Stage 2.

4.4. Block stage_2

Một số key quan trọng:

- `target`: chỉ dùng để suy ra backbone NNX tương ứng;
- `ckpt`: có thể là file PyTorch `.pt` hoặc thư mục Orbx;
- `input_size`, `patch_size`, `in_channels`;
- `hidden_size`, `depth`, `num_heads`;
- các toggle như `use_rope`, `use_rmsnorm`, `use_swiglu`, `wo_shift`.

4.5. Block guidance

Hai mode chính:

- `cfg`: adapter dùng chính model làm conditional và unconditional pass, với nhãn null mặc định bằng `num_classes`;
- `autoguidance`: adapter load thêm `guidance.guidance_model` làm guide network.

4.6. Block training

Một số key được ánh xạ trực tiếp:

```
training:
  global_seed: 0
  epochs: 1400
  global_batch_size: 1024
```

```
ema_decay: 0.9995
num_workers: 4
log_every: 100
ckpt_every: 5000
sample_every: 10000
base_lr: 0.0002
final_lr: 0.00002
beta: [0.9, 0.95]
wd: 0.0
schedule_type: linear
```

Lưu ý:

- nếu config chỉ cho epochs, adapter sẽ suy ra total_steps từ num_train_samples;
- optimizer mặc định vẫn là AdamW;
- scheduler được chuẩn hóa về các mode đơn giản như constant, linear, cosine.

4.7. Block eval

Đường JAX hiện dùng phần FID nhiều hơn phần validation loss. Nếu fid_ref là file .npz của repo cũ, adapter sẽ tự chuyển nó sang định dạng pickle mà backend NNX hiểu được.

4.8. Override từ CLI

Các entrypoint JAX cho phép override nhanh bằng:

```
python3 src_jax/train.py \
  --config <config> \
  --set training.global_batch_size=256 \
  --set guidance.scale=1.5
```

Mục tiêu là giữ đúng một schema YAML, nhưng vẫn cho phép chỉnh nhanh như CLI.

5. Vận hành, artifact và FID

5.1. Sampling Stage 2 trên đường JAX

5.1.1. Sampling nhanh

```
python3 src_jax/sample.py \
  --config <sample_config> \
  --seed 42 \
  --class-labels 207,360 \
  --output sample_jax.png
```

5.1.2. Sampling phân tán

```
python3 src_jax/sample_ddp.py \
  --config <sample_config> \
  --sample-dir samples_jax \
  --num-samples 50000 \
```

```
--per-proc-batch-size 4 \
--label-sampling equal \
--fid-ref /path/to/reference_stats.npz
```

Artifact đầu ra:

- các file PNG theo index;
- các shard .npz theo rank nếu bật `-save-npz`;
- giá trị FID in ra terminal nếu cung cấp `-fid-ref`.

5.2. Build reference statistics cho FID

```
python src/build_fid_stats.py \
--input /path/to/reference_images \
--output /path/to/reference_stats.npz \
--device cpu \
--batch-size 64 \
--num-workers 32 \
--num-threads 96
```

Đường JAX vẫn tái sử dụng được file `reference_stats.npz` này.

5.3. Artifact của Stage 2 training

5.3.1. Đường XLA

Một experiment trong `results/` thường có:

- `log.txt`,
- `checkpoints/<step>.pt`,
- `fid_eval/step_<step>/...` nếu bật online FID.

5.3.2. Đường JAX

Một run trong `results_jax/` thường có:

- `jax_adapter_config.json`,
- thư mục checkpoint Orbx do backend quản lý,
- media và scalar trên wandb nếu bật `-wandb`,
- toàn bộ workdir có thể được đẩy lên Hugging Face nếu truyền `-hf-repo-id`.

5.4. Đẩy model lên Hugging Face

```
python3 src_jax/push_hf.py \
--path results_jax/<run_name> \
--repo-id <hf_user_or_org>/<repo_name>
```

Hoặc gán thẳng vào lệnh train:

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

5.5. Các lưu ý vận hành quan trọng

- Đường JAX cố ý giữ nguyên schema YAML của repo để giảm chi phí bảo trì.
- `stage_2.ckpt` trên đường JAX có thể là PyTorch hoặc Orbx, nhưng cách load sẽ khác nhau.
- Online validation loss hiện có parity tốt ở đường XLA hơn đường JAX.
- FID trên đường JAX và XLA đều dùng reference stats của repo, nhưng backend JAX sẽ chuyển `.npz` sang pickle nội bộ khi cần.
- Nếu chỉ cần inference Stage 1 ở JAX, dùng `src_jax/stage1_sample.py`.
- Nếu chạy trên Kaggle TPU v5e-8, dùng notebook `raes-jax-celeba-kaggle-tpuv5e8.ipynb`; notebook này cài `jax[tpu]` và kiểm tra `jax/TPU` bằng tiến trình Python mới để tránh notebook-kernel skew.
- Nếu cần huấn luyện Stage 1 kiểu GAN đầy đủ, hiện vẫn phải xem đây là phần chưa được JAX hóa trong branch này.

5.6. Checklist thực tế trước khi chạy dài

1. Xác nhận `stage_1` và `stage_2` khớp latent shape.
2. Xác nhận dữ liệu train và val đúng dạng `ImageFolder`.
3. Build `fid_ref` trước nếu muốn FID ổn định.
4. Export đủ biến wandb trước khi bật `-wandb`.
5. Nếu muốn upload lên Hugging Face, xác nhận token nằm trong biến môi trường được chỉ định bởi `-hf-token-env`.
6. Với JAX sampling số lượng lớn, chọn `-per-proc-batch-size` đủ an toàn theo bộ nhớ thiết bị.